Namaste Ai Clean Digital Notes

Hi, I'm Ashutosh **3**
Explore My Digital World 3
Welcome to the Namaste AI Notes **4**
How to Learn 4
AI Evolution: From Turing to Agentic AI **6**
What is Artificial Intelligence? 6
History of AI 6
1950-1980 Rule-Based AI 7
Rise of Machine Learning 7
Deep Learning 7
Computer Vision Revolution 8
NLP Natural Language Processing 8
Transformers (2017) 9
Large Language Model (LLM) 9
Generative AI 9
The ChatGPT Moment Nov 2022 9
AI Today 10
Agentic AI 2025 10
Industry Next Actions 10
Want me to write notes on a course? 10
Have a notes collection to feature here? 10
ChatGPT vs Google: How LLMs Actually Work — Training, Inference, Hallucinations & RAG **11**
When we search on Google 11
When we prompt on ChatGPT 11
What is Google indexing 12
How LLMs generate responses? 12
What Knowledge Does LLM Contain? 13
Knowledge cutoff 13

Base Model 13
Inference vs Training 14
Why models are fake fluent? 14
Confidence as evidence 15
How tools extends the model? 15
Does the mode have self awareness? 15
Want me to write notes on a course? 16
Have a notes collection to feature here? 16
LLM Tokenization: Tokens, IDs & Common Misconceptions 17
Vocabulary & Token ID 18
Byte Pair Encoding (BPE) 18
English vs Other Languages 19
Tokenization fertility 19
Context window 20
Want me to write notes on a course? 21
Have a notes collection to feature here? 21
LLM Embeddings Explained: How AI Understands Meaning & Context 22
Vectorization 22
Embedding 23
Embedding as coordinates 23
Semantic similarity 23
COSine Similarities, ... 24
Token Enmbedding 24
Positional embedding 25
Token vs Text Embedding 25
Misconception 26
How LLMs Work: Neural Networks, Transformers & Attention Explained 28
GPT in ChatGPT 28
Attention is all you need! 29

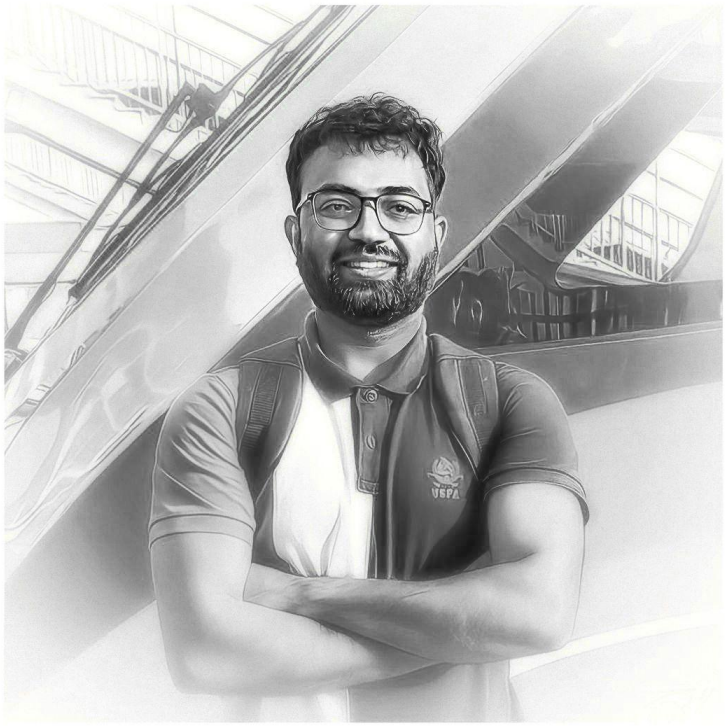
7 articles from **heyashu.in**.

heyashu.in · 1 page · 1 min

Hi, I'm Ashutosh Anand Tiwari

Welcome to my digital space where I share my journey through code, creativity, and exploration. Here you'll find my thoughts on technology, travel experiences, and various other interests cultivated through my digital garden.

heyashu.in



heyashu.in · Ashutosh Anand Tiwari · 2026-08-15 · 2 pages · 3 min

Welcome to the Namaste AI Notes

Welcome to the course. Let's discuss the mission of this series. As we know, AI is everywhere.

Every day, we are using AI in each task, and when we don't use it, we feel like something is missing, haha. This is the line I have added.

We, as engineers, managers, and people in tech, should learn what AI is, how to use AI, and how to solve our problems using it. But how do we stand different from others? We need to read and study deeply about AI.

So, let's start. Let's solve existing problems using AI. Let's learn how to build real products with the help of AI. There are hundreds of topics related to AI — Gen AI, ML, Neural Networks, and many more. We will learn what is actually necessary.

Let's talk about the roadmap.

Intro → LLM → Prompting → AI for SDEs → AI Tools → RAG → AI Agents → MCP → AI Engineering → Projects

We will go deep into the topics, so don't get bored. There is nothing like spoon-feeding, but we will have fun while reading these notes. So keep patience and keep yourself hydrated.

We are moving with **JavaScript / Python** here, but there is no language barrier. Use your AI brain to convert your code. Basic coding knowledge will be enough for us to learn.

We will also touch **Backend, Express, MongoDB**, and more.

So, all we will do in this journey is take a long breath and let's go.....

Create a community. Post your journey on social media. Take challenges. Make friends. Learn from friends. Learn how to get the job and see how others are learning and finding jobs.

Let your social media posts reach recruiters as well. So, be active on social media too.

How to Learn

Don't try to binge-watch. Take breaks. Use the **Pomodoro technique**.

Make notes. I personally recommend not watching a video for more than **50 minutes at once**. Take at least a **5–10 minute break**. This completes around 1 hour of learning, and then continue the same way.

Use Pomodoro extensions in your favourite browser.

Don't try to jump between topics.

Help others in the community.

Break things while learning. Learn and be curious while learning. Be experimental. Don't wait for a teacher to teach you everything. Play with it. Go and explore on the internet. There are many examples and many people who have already done things before.

Watch a video 2–3 times if you don't understand a topic. Keep patience.

All comes on time, but you have to keep taking action.

Remember Krishna:

"Do Karm, fruit will come to you automatically."

Want me to write notes on a course?

Tell me the course, book, or research topic you want covered. If it helps learners, I'll consider adding structured digital notes for it in the garden.

Have a notes collection to feature here?

Do you have open notes you want featured in this digital garden? Let me know — we can collaborate and publish them for learners worldwide.

heyashu.in · Ashutosh Anand Tiwari · 2026-08-16 · 5 pages · 11 min

AI Evolution: From Turing to Agentic AI

Now let's discuss from where it began — the seed before turning into the vast forest.

If you look at the top 10 companies having the largest market cap in the world, 99% of them are just investing lots of money in AI. Who will go first to win? Who will release a new model in an efficient way, and much more. So much research is going on, Coding Gen, Video Gen, Image Gen, so in all areas they are trying to be the hero. So why are we lagging? As students or as working in an organization, let's learn and be the hero.

So here let's discuss about where it all started.

What is Artificial Intelligence?

When we say let's make machines intelligent, like recommending movies, driving a car, writing poems, making my notes from the video I learned it from in my way, generating an image where I am with Salman Khan, these are intelligence or work we as humans do, and when a machine, a computer, or system algorithm does this then it is called Artificial Intelligence, because it is not manual. It did not really happen artificially; a machine is now capable of doing this.

```
So AI is a science of making machines
perform tasks that normally require human
intelligence.
```

Don't think AI will read all notes and courses and make you land in a job, no no, you will be always there, so keep learning haha. Joke apart, let's continue....

History of AI

So computers existed long back, around 90 years ago also we had computers where we do things.

It all started in 1950. That time many scientists were doing research: can a machine think? Because computers were there performing things. So there was a mathematician named Alan Turing who started researching and making machines capable. Can a machine think? He developed a Turing Test. Let me explain what a Turing Test is.

Suppose there are two rooms, Room 1 and Room 2, and one more room, a Judge room. Where humans exist and those two rooms have one machine and one human. So the judge, who is human, asks the same questions to those two rooms, from the machine and human, and gets one answer from those two rooms. The judge needs to decide who is human and who is machine based on the answers. If the judge is not able to do this, then that means the Turing Test is passed.

But in 1950, there was no AI word. By 1956, John McCarthy, a computer scientist, named the word Artificial Intelligence. Then some researchers came together to see the possibilities and with an ambition believed:

```
Every aspect of learning and intelligence
could in principle, be described
precisely enough for a machine to
simulate it
```

There was so much roller coaster in the history. There was an AI winter also, where no one was talking about this from 1987-1993. 1986 synthetic learning, 1997 — this question raised again. Synthetic intelligence research was about machines learning from itself, and different things were going on.

Read this blog as well: Synthetic intelligence — alternative or future of AI

Currently, the situation AI is going beyond the human intelligence. Is it synthetic intelligence? We are using the word AI, but this is all synthetic intelligence. We are going beyond. ChatGPT, we use it, it's real, not fake, name is artificial but it is possible. Now you check and learn about it, let's move next.

In Deep Blue, [IBM Software] defeated Garry Kasparov, AI chess champion. Have machines become smarter than humans? Deep Blue was not an Artificial Intelligence but a system that was designed based on permutations and combinations of what can be the best possible move in chess. But that was a big thing happening after 30 years ago. We are taking that example.

1950-1980 Rule-Based AI

Intelligence was simply a collection of rules, 100s of if-else rules, that make a machine decide things.

For example, detection in a mail body: if a lottery dollar FREE, then mail can be spam. So based on conditions, a machine or algorithm can decide the next step to flag anything, and these machines are known as Expert Systems, and then there was AI winter.

Rise of Machine Learning

Till now everything was rule-based to make machine intelligence, but that was not working way. Anyone can write things in a different way to do the spamming, the rule-based things will fail. Not possible to write all conditions. That's why machine learning came into the picture.

Like machine can identify the difference between dog and cat. Machine can identify. A machine is trained on 10 lakh images of dog and cat, then machine can understand now based on training data given to them, based on dog and cat. So machine was able to make prediction.

Like give an animal pic, machine can decide whether that is cat or dog, so machine started learning patterns. So we provide training data, but human observation is required. Machine learning was not enough for all possibilities.

Deep Learning

This was the concept of neural network. Can machine think and learn themselves? So scientists started learning and researching on human brains, and human brain has neurons, and the way we think can machine also think? What if we can make connection like neurons of information, called Deep Learning.

This was the breakthrough. Can computer recognize a cat, recognition of faces, speech recognition based on tone and all, phone face recognition. Like machine can predict based on pattern.

This was all possible because of deep learning. There was skilled training as well, but it was different kind of neural networks, RNN, KNNs also, right? So there is chapter, if you have time you can learn. If you want to learn, go check ML courses and books are there. Here we are focusing on history and usage of that.

Deep learning leads to GPU revolution, Internet became vast, and it helps to collect data, and neural network can take more data now to train, so better algorithms. And less data on the internet and in bucket was also a reason of winter of AI.

```
So Deep learning is a subset of ML
focused on teaching computers to learn
and make decisions by processing data
through neural network inspired by human
brain.
```

Computer Vision Revolution

In parallel, Computer Vision was also taking shape. There was ImageNet, and in 2012 a large neural network got trained on ImageNet dataset. I read the full ImageNet database contains **over 14 million** high-resolution images divided into more than 20,000 categories. In 2012 Alex Krizhevsky & team built AlexNet, a deep neural network. It made machine identify images, image recognition. As a result of this, face unlock, self-driving cars, X-ray recognition, shopping app identifies products, and as a result machine can see.

Now overall we had that capability. I know this deep learning, why called deep, you can search online and get the result. Here we are just getting familiar with the term and thoughts. So yeah, let's continue.

NLP Natural Language Processing

People used to think text data is easy and other types, image and audio and video are complex data sets, but because text itself a large data set, it's hard to understand and crack and train upon. Let's take an example, read this sentence:

`I saw a man with a telescope`

So there is two meaning of this sentence, I saw a man and that man has a telescope, or I saw a man but I am using the telescope. So it's confusing.

2nd example is:

`River bank` and `Bank of India`

Now machine can also get confused. The same word bank refers to different things in just two words, so the English sentence itself is tough for a human as well, machine had to use to make that training.

So context differs, meaning differs. But nowadays ChatGPT and new-age models understand everything, so what we see is result of time also.

In NLP, Bag of Words, n-gram, RNN (Recurrent Neural Network), LSTM (Long Short-Term Memory) are theories and also techniques used for NLPs. RNN was the major breakthrough

that made large sentences make machine understand, and later LSTM made that more enhanced, understanding more than a page. But here was issue with 100s of pages, connecting dots was very tough.

Transformers (2017)

Transformers are the most remarkable creation of history of mankind.

ChatGPT, DeepSeek and whatever new AI things you see, it's all possible because of Transformers only, and this is one kind of architecture we will discuss properly. The Google engineers researched hard and released a research paper named `Attention Is All You Need`, and whatever model, GPT model you see, all based on this concept.

Context and Gen AI is all possible because of this architecture only, so 2017 research changed everything.

When we provide example: Lion did not cross the river because `it` cannot swim, and 2017 was the year when model started reference and context of the sentence. In this case, it is referring to the lion, dots started connection. LLMs were only possible because of this.

Read research paper here: Attention Is All You Need (PDF)

Large Language Model (LLM)

LLM is nothing but a transformer which is trained on very, very, very X100 large data, that understands your query. LLM was only possible because of large data sets, GPU machines, heavy systems, computation power needed to train that.

So data and computation power, GPU is two main factors having a LLM model. And countries which are lagging, having their own AI, they are weak on LLM, because it's cost intensive, not many countries have so much resources. Check on internet how much cost it includes to make this possible.

Generative AI

In earlier days AI had different sections, like this AI can do Classification, this type of AI can do Prediction, this LLM or AI is only doing Recommendation. But modern time, AI is able to generate things, like poems, songs, fully copyright-free content, like new images, videos, the faces that we never assumed and never existed on earth.

So Gen AI is nothing but a model that generates the thing. And a multimodal model generates multiple things, Documents, videos, audio, text and images, and this is what the concept of Generative AI is. Before, we had to just relate things and give to us. Nowadays we can write a full code of a project using our idea. So it's not limited to helping only, doing generating also.

The ChatGPT Moment Nov 2022

Sam Altman released ChatGPT in Nov 2022 and revolution happened. Machine can do conversation, we can chat, and in 2022, public started using it. 2017 to 2022 all were like who will re-

lease, make these models available to the users, and in 2022 Sam Altman did this.

And ChatGPT becomes the most downloaded app in world history, and still race is going on, who will create the best model for our use case, and we as engineers continue that use case, making our work easy.

We will discuss about RLHF and Memory later. Memory is nothing but context, how much our conversation is remembered by the model we are using.

So ChatGPT was the first. After that Gemini came, Grok came, DeepSeek came, Claude came.

AI Today

Earlier no AI could do so many things. Everything what we could do before:

Thinking, Call API, Plan, Write API, Understand, Remember, Search web, Working autonomously.

Use tools, complete tasks, write code, debug code, deploy, test it and so many things.

But this is not something you need to worry about and learn so that nobody will replace you. Now we are superpowered with things, so don't sit, else someone else will go ahead of you.

Agentic AI 2025

Now we are living in 2026 and last year it was all about Agentic AI, and all we will discuss things in details in this later.

Let's recap the things:

1950 – Alan Turing

1956 – John McCarthy

50s–80s – Rule-Based AI

1997 – Deep Blue defeats Garry Kasparov

90s – Machine Learning

2000s – Deep Learning

2016 – AlphaGo beats Lee Sedol: watch this video for more

2017 – Transformers (Attention Is All You Need)

2025+ – Agentic AI

There is also one document by DeepMind by Google, AlphaGo, you can watch:

Industry Next Actions

And we are applying to every possible industry.

That's all we see. See you later. Thanks.

Want me to write notes on a course?

Tell me the course, book, or research topic you want covered. If it helps learners, I'll consider adding structured digital notes for it in the garden.

Have a notes collection to feature here?

Do you have open notes you want featured in this digital garden? Let me know — we can collaborate and publish them for learners worldwide.

heyashu.in · Ashutosh Anand Tiwari · 2026-08-19 · 6 pages · 12 min

ChatGPT vs Google: How LLMs Actually Work — Training, Inference, Hallucinations & RAG

Have you ever thought about where ChatGPT gets answers for us? Let's do an operation of ChatGPT and LLM. Let's see whether ChatGPT knows everything, does it live search, or is it just a guess? Is it artificial or synthetic? Now people are moving from Google Search to ChatGPT. Is there any chance of misinformation? What's the difference between Google Search and ChatGPT? We will explore everything here. Here we go.

In ChatGPT, nowadays people are using it more than Google because it gives us direct answers without going to separate pages, where our tab was getting filled and the computer was getting slowed. We remember those Stack Overflow days.

Okay, back to ChatGPT. You can ask anything to ChatGPT, and it will answer if that question does exist. If suppose anything doesn't exist, it will simulate that thing exists somewhere. You will not get proof of that. The same question, if asked in Google Search, See the image for reference.

Now the question is, Google Search is kind of filtered results, but GPT is giving us a ready-made answer created by itself. The question is why and how this happens. We will discuss it all.

When we search on Google

When we search on Google, Google takes the question, searches in its index, and ranks relevant documents and returns the result.

Now the question comes, what is an index? Index is nothing, just a book content list. When you want to read something, you go and search in the index instead of going through the entire pages. So index is just a reference of webpages.

This is how Google works normally when you search.

When we prompt on ChatGPT

So when we put a prompt in ChatGPT input, ChatGPT has an encyclopedia of knowledge. It has learned patterns and knows all that the content contains, and based on the prompt, it will provide a response that is developed by some other pattern. No, ChatGPT will provide you its own version of the answer, with reference if needed. But it is something like if you have read your book or you watched the movie Bahubali, if someone asks why Katappa killed Bahubali, you will go and remember the movie content and you will give your generated answer, not exactly what happened at that time.

So summary is, ChatGPT has learned some pattern, and it generates an answer for you. Google, whereas, searches on the index and retrieves the answer.

I think informally it's clear now. It's all about Retrieval vs Generation. Having an answer as "I don't have this answer" is possible for Google to answer, but ChatGPT will not do this. It can generate some random thing.

What is Google indexing

Google indexing is **the process where Google stores and organizes data from web pages into a massive digital database**. When you look for something online, Google searches this database instead of the live internet. But how does Google manage things with the vast amount of internet data? So Google has a concept of crawlers or spiders. Actually, this is with all search engines, and it keeps searching all databases across the internet and keeps the index list updated. So if something we search for has just happened 1 min ago, we will get the result on Google because Google spiders have already crawled it in seconds, and that index is top-ranked, and that's why we get the result in Google Search.

Now the question is, does the crawler rank everything? The answer is no, because there are various rules for crawlers to identify a webpage, like Domain Authority, page speed, keywords, average time spent (retention), backlinks (other sites giving a reference to your website), meta tags, date, and SEO things. Google's algorithm for this task is not open source; that's private, and these regular best practices are used to make that possible.

Let's discuss some more facts about Google vs LLMs.

Search engines don't guarantee the truth. They can be outdated, and rankings can be imperfect. The relevant information can come as the 4th result, and some irrelevant results can come on top, and that leads to misinformation. But there are positives also. We can check the website from where this information came. Now suppose a new channel XYZ posted something new, and if you don't love that website, you can ignore it, because the result which is on Google is from some website only.

Let's discuss LLMs in a shorter time. Here we can't see the source unless we ask. We can't see the data or information because it's generated, so the information can be misleading.

How LLMs generate responses?

As we discussed before, LLMs predict and generate. So when we ask anything, it sees the possible words and content that can be replied.

So for example:

The sun rises in...

So what could be the next word? Like here, **the east**. So it connects the words. How is it generating? Where does it get the words?

Like **Roses are** __, beautiful, red, blue, etc. Based on the training of the specific model, it will generate the word. It's a guess based on learning from data. It's kind of autocomplete in a smarter way, where which word will fit in automatically is all about the probability of the next word. We'll discuss that later in depth.

To add more things, LLMs are trained with multiple languages, grammar and rules, rea-

soning facts, stories, associations between places, events, and ideas.

What Knowledge Does LLM Contain?

It's all trained on a neural network that contains a very large collection of numbers called parameters or weights. All the companies scrape internet data, they read all the data present on the internet, and that forms a pattern, and that helps us get the result.

It's not that simple, I am saying that. A lot of patterns form, and many complicated algorithms work behind the scenes.

So when we say **Rose**, the combination of words comes as a prediction: beautiful, red, blue, etc. So it's all based on patterns and probability scores.

Knowledge cutoff

Every LLM has a knowledge cutoff date. Now, I mean LLMs are not trained every day, so the data is trained on data that is still available up to a certain date.

The question comes: till what date is it trained? All LLM models tell that in their documentation.

But why then does it provide us with current knowledge? These open questions come to us, so we will discuss that too later.

But remember, training with data is a complex thing and is price-heavy, so that's why it comes with a knowledge cutoff.

Base Model

When a new model is trained to predict, a base model is created by a company. A base model is the model that predicts the next token or word.

A **base model** of an LLM is **the raw, core artificial intelligence**. It has read massive amounts of text from the internet, books, and articles. Its only actual job is to guess the next word in a sentence. Think of it as a giant, highly advanced autocomplete tool, but it is not yet trained to act like a helpful chat assistant. Read this: https://toloka.ai/blog/base-llm-vs-instruction-tuned-llm/

So when we see modern ChatGPT, it is built on top of a base model, which has so much more access and is super-powered. So, on top of the base model, things like tool access, human feedback, security/auth, web search, guardrails, content filters, conversation management, system instructions, etc. are added, and it becomes an AI assistant like ChatGPT, Grok, Claude, and Gemini.

The job of the base model is to predict the next text, nothing more than that. So ChatGPT and all of these are built on their base models. The base model is not restricted, so you can ask anything without filters. That's why companies never expose the base model directly. But there are some which are open source; check them, you will find some.

So, taking an example: **Car is ChatGPT and Engine is the base model.**

See the companies and their base model names:

OpenAI: GPT-5, GPT-4o, o1, o3 (Reasoning)

Anthropic: Claude 4, Claude Sonnet 4.6, Opus

Google: Gemini 3, Gemini 2.5 Pro, Gemma 3 (Open weights)

Meta: Llama 4, Llama 3.1 (Open weights)

DeepSeek: DeepSeek-R1, DeepSeek-V4 (Open weights)

xAI: Grok 3, Grok 4.3

Alibaba: Qwen 3.5, Qwen3

Amazon: Nova (Pro, Lite, Sonic)

Inference vs Training

This is one of the more terms people use so much, and let's see how it is different from training.

Inferencing is something that happens after training. When we ask a prompt to an LLM model, the process of giving me an answer back is called inferencing. The process of inferencing is giving you an answer.

In easy words, **training and inferencing are the two main steps in how artificial intelligence works. Training is the school phase where the AI studies huge amounts of data to learn rules and patterns. Inferencing is the working phase where the ready AI uses that knowledge to answer questions or make real-world decisions.**

Why models are fake fluent?

Do you remember in our life we have a proverb:

```
Tez bolne se koi baat sahi nahi ho jati.
```

Same applies here. When LLMs respond to something confidently, it does not mean it is correct. So fake fluency is not truthfulness, and this is called hallucination.

An AI hallucination is **when an artificial intelligence makes up information and presents it as a hard fact**. The AI is not trying to lie; it just guesses the next best word to complete a sentence, sounding very smart and confident even when it is totally wrong.

That's why we say medicine and all should not ask ChatGPT and believe it. When we ask the Gemini voice agent, it says, "I'm not a doctor," so it's an assistant, not a doctor. Remember, language quality and factual accuracy are separate dimensions.

But why does this hallucination occur? What is the reason? That is because of insufficient knowledge. The reason can also be ambiguous information, no new data, false assumptions, unreliable patterns, and models being optimized to answer. It can create an elephant with two legs because it's optimized to answer.

Some base models sometimes will not give me the right answer, like math problems, but ChatGPT, the instruction-tuned model, will give you the proper answer.

As we discussed, models are optimized to answer, but sometimes it will not answer you and will say, "I don't know." There might be questions that lead the model to answer "I don't know." Like if you say, "Which React course does Ram Lal from ABC District teach online?" then it will not answer you. It will say, "I don't have reliable information about this."

So there are different reasons: weak pattern systems, instruction, tool requirements, prompt wording, etc. These all can lead to the model not answering instead of providing an optimized answer.

Confidence as evidence

Humans often use tone as evidence. Sometimes we say, "This answer is XYZ, not ABC," then the answer is definitely XYZ. When someone is very rigid and confident, we think the answer is correct. Don't connect this with any politician.

`Confidence as evidence` means an AI uses how sure it feels about an answer to decide if that answer is true. If the AI is very sure, **it treats its own guess** as proof.

How can we remove this wrong confidence? Ask for proof, ask for the source of the response, say, "Only answer if you are sure," and ask for uncertainties. Then the LLM may say, "Yes, that was wrong," etc. This is one way to reduce overconfident answers, but it doesn't guarantee correctness. Using grounding and allowing web search can provide external evidence to verify the response.

How tools extends the model?

See, tools are like **superpowers** for LLMs. They can be used for custom use cases and make all kinds of things possible, for example, tool calling, your own API, your own database, email, calendar, location, weather, code execution, internal documents, files, and many more.

Because without tools, the LLM does not have access to your files, your location, your emails, your database, etc. So, for a personal assistant to actually be useful for a particular user or group, **tools are required**, and that makes the LLM much more powerful.

So, if you see, when we mix **web search + LLM**, it becomes more powerful. Retrieval provides **external evidence**, and generation gives you a beautiful, natural-language answer. This combination is called **RAG — Retrieval-Augmented Generation**.

So this is one of the most important concepts for personal work and real-world AI applications.

Does the mode have self awareness?

Yes, some information you can get: number of parameters it is trained on, knowledge cutoff, where it is deployed, who owns it, and all the other information we should know can be answered by ChatGPT or any LLM.

But it's not fully self-aware, so when you ask anything, there are a few sources it can have:

- Training dataContextSystem prompt / instructionsTools

So the source can be training data, on which it is already trained; context, which is what we are discussing; system prompt, which tells it what we ask and how it should behave; and tools, which are like superpowers.

These are what can turn an LLM into a more capable, instruction-tuned assistant instead of

just a base model. The model itself generates based on the information available in its current context; tools and external data can provide information or actions that are not inside the model's training data.

Okay, that's all about the basic terminology and how LLMs are different from Google and Search. We will also go deep and learn more about this.

So, stay with me. Have a good learning. Signing off. Bye-bye. Share with your friends. Thank you

Want me to write notes on a course?

Tell me the course, book, or research topic you want covered. If it helps learners, I'll consider adding structured digital notes for it in the garden.

Have a notes collection to feature here?

Do you have open notes you want featured in this digital garden? Let me know — we can collaborate and publish them for learners worldwide.

heyashu.in · Ashutosh Anand Tiwari · 2026-08-21 · 5 pages · 9 min

LLM Tokenization: Tokens, IDs & Common Misconceptions

Now let's understand the input of an LLM. Does it understand only English? But we see that it also responds in different languages. We know programming languages are used because computers don't understand English, that's why we have specific languages to make computers understand, like C and C++. So what about LLMs?

Computers don't understand English words, that we know 100%. If we are saying, **"These notes are awesome,"** this sentence cannot be directly understood by an LLM. What an LLM understands is numbers.

Let's see how. So when we say **"These notes are awesome,"** it gets broken down into different words, like:

`These` `notes` `are` `awesome`

And these words are converted into numbers, and an array of numbers is passed to the LLM, like:

`[78, 12, 23, 433]`

When these sentences are broken into words or pieces, they are called **tokens**.

The number associated with a token is called the **token ID**, and the sequence of tokens is what the LLM understands.

But the question comes: if an LLM knows the numbers, then how does it predict the next word? So instead of predicting the next word directly, the LLM predicts the next **token ID**, which then maps back to a token. This is an informal way to understand the secret language of LLMs.

Breaking a sentence or unit of text into tokens is done by a **tokenizer**.

There are multiple types of tokenizers available that companies use. Different models and different types of tasks can use different tokenization methods. This is one of the most important parts of LLMs, used during training and inferencing. You can learn more about tokenizers online.

One more thing: it doesn't guarantee that one word becomes one token. It is possible that:

is converted into two tokens, like:

So, one word is not always one token.

Okay, okay, so much theory. Let's see these things live on websites. These are two websites you can check to see how it works:

https://platform.openai.com/tokenizer https://tiktokenizer.vercel.app/

Now here I have put our statement. Here is how the output comes:

On the website, you can see you can use different tokenizers, and that generates different token IDs and breaks the text in different ways.

But the question comes: what are these numbers, and why are these random numbers?

Actually, when a word is converted into smaller parts, it is called **subword tokenization**. The question is, why do we need this?

Subword tokenization depends on the tokenizer and the vocabulary it has learned. If a word or piece of a word is very common, it may become one token, but a larger or less common word can become multiple subword tokens.

Like, see the above screenshot: `untrustable` becomes 3 tokens because parts like `un`, `trust`, and `able` can represent meaningful patterns and can be reused in other words. This helps the model handle many different words without needing every possible word to exist as a separate token.

I think now you are able to understand it.

But I saw something where I was confused: `undone` becomes only one token. I was expecting it to break into two tokens, like `un` + `done`. So why did that happen?

The answer is that **tokenization is not always based on grammar or the meaning of a word**. The tokenizer does not necessarily split a word into its meaningful parts. If `undone` already exists as a common token in that tokenizer's vocabulary, it can be represented by a single token. The same word can also be split differently by another tokenizer.

So, **one word is not always one token, and one prefix + word is also not guaranteed to become multiple tokens. It depends on the tokenizer vocabulary and how that tokenizer was trained.**

Vocabulary & Token ID

Every tokenizer has a vocabulary. This is the way each token gets matched to a token ID. And how does that happen? Using the vocabulary mapping.

So, in easy words:

A tokenizer's **vocabulary** is the fixed list of all unique words, subwords, or characters that an AI model or text processor recognizes and can convert into numbers.

So, many companies use different types of tokenizers, as we discussed. OpenAI models use different `Byte Pair Encoding (BPE)` tokenizers, categorized by their different vocabulary sizes. Meta Llama uses `SentencePiece-BPE` frameworks. Read more online; it's not possible to write everything here.

So, understand that the model has a vocabulary where tokens like `un`, `trust`, and `able` are mapped to numbers using the vocabulary.

Byte Pair Encoding (BPE)

This is one type of tokenizer, and this is used by OpenAI. Here, what we want to understand is, suppose there is one word, `low`, and it has `15` as the number assigned. Now it can be used for `low`, `lower`, and `lowest`. The same goes for `new`, `newer`, and `newest`, and

also for `un`, `unbreakable`, `untrustable`, `unmanageable`, etc.

So, using BPE, we can achieve this. Each token can be represented using bytes. You know a byte can be represented by 8 bits, I mean `0` and `1`. So these combinations of bits represent numerical values. The tokenizer starts from smaller byte-level units and learns which neighbouring pairs commonly occur together. These pairs can then be merged into new tokens, and the vocabulary size increases.

So it's all about **merging neighbouring pairs**. Common combinations can become a single token, which is why the same smaller pieces can be reused across many words.

There are different tokenizer methods also used, like `WordPiece` and `Unigram`. Please read about those online and just understand vocab size make tokenization very good and effiencient

English vs Other Languages

I am learning Artificial Intelligence , मै Artificial Intelligence सीख रहा हूँ.

Or if we write in Hinglish: **Mai Artificial Intelligence seekh raha hu** or mix of that, so don't think it is all tokenized the same way because the meaning is the same. No, there is no meaning of words that LLM understands, and tokenization happens based on the text. There is no emotion, so different languages can have different token IDs. Remember it.

Now you can see in the image, with the older version, it was broken into the same line into so many tokens, but in the modern tokenizer version, it uses fewer tokens. So vocabulary size increases, and new languages are more efficiently supported by OpenAI. So vocabulary is the main factor in understanding how a model handles different languages.

But English is more efficient. See the above image: the same thing in English takes fewer tokens, but Hinglish is more costly in terms of tokens.

But it's crazy that it understands Hinglish so well, because in Hindi we write a word in different ways, like **main, mai, maii**, and it can be written in multiple ways.

Tokenization fertility

Tokenization fertility is **the average number of tiny text pieces (tokens) an AI creates for every single word**, meaning how many tokens are produced from a word.

Lower fertility means the AI uses fewer pieces to represent a word. This makes the AI faster, cheaper to run, and able to process more text at once. [1, 2]

Higher fertility means the AI chops words into lots of tiny fragments. This uses more tokens, increases the cost, and fills up the context window faster, especially in some non-English languages.

How do LLMs understand emojis, code, special characters, capital vs small-case letters? Let's understand.

Emojis and characters are also mapped in the vocabulary, so everything is properly engi-

neered, and that gives us the related result. Spaces also change the token ID, so it's all about mapping.

Check any tokenization playground, like the OpenAI tokenizer or any other tokenizer available online. You will see that adding or removing a space, using curly braces, adding dashes, or removing dashes can all change the tokenization and token count.

Each text pattern has its own token ID and mapping.

Okay, there are some special tokens also added by the LLM when we do a prompt. Those are from the system instructions. What are these system instructions? With each model, all we provide system instructions.

We will learn later about system tokens, the start of the message, the end of the message, and how these are based on the thing we are writing, like a text document, etc.

A **system instruction** is a hidden rulebook that tells an AI how to behave, while a **user instruction** is the specific question or task you type in.

User instructions are **simple, clear steps that tell a person how to use a product, app, or tool.**

So, the way these system and user prompts are passed to an LLM is different for each LLM model and company.

Context window

You remember when you chat with an AI assistant and you come back after 7 days, it also remembers what you chatted about before 7 days, and you keep the project and all the chats under that. This is all possible because it keeps the context, or we can say memory, of the chat.

So, each LLM has a **context window**. In that window, it remembers your chat. A context window is the amount of tokenized information a model can process within a request or active generation context.

In easy words, a **context window is the short-term working memory of an AI, measuring the maximum amount of text it can "see" and remember at any single moment.**

But context includes the **system instructions, documents, text, tool outputs, everything**, not just what you are typing.

The context window is shared between what you send and what the model generates.

So what happens if the context window overflows?

So, first of all, to avoid it, **truncate your input**, summarize previous messages, don't put the entire thing as one input, and always ask for the relevant things that you need.

Split the work into different chats, module-wise, not everything in one chat. **Work A on Chat A, Work B in Chat B.** Do not assume AI always remembers your requests, which are very old. It remembers the summary, but only while the context is live!

Context doesn't mean chat history, so never think that.

Remember, a longer prompt does not always give you a better result. **Tokens directly affect API cost.**

There are also tools available where you can input your big prompt, and they will provide you with a polished prompt. That will make you more efficient and cost-effective.

So, just list some misconceptions people have:

- **One token is equal to one word:** This is wrong.
- **Everyone uses the same tokenizer:** This is also wrong.
- **Token ID represents meaning:** Answer is no.
- **One visible emoji is one token:** Answer is no.
- **A larger vocabulary is always better:** No. (If I say yes, I need to search why.)
- **Larger context means perfect memory:** Not necessarily.
- **More tokens always lead to better results:** No.

Ok so thats all for this one, will see you in next one, Bye Bye !

Want me to write notes on a course?

Tell me the course, book, or research topic you want covered. If it helps learners, I'll consider adding structured digital notes for it in the garden.

Have a notes collection to feature here?

Do you have open notes you want featured in this digital garden? Let me know — we can collaborate and publish them for learners worldwide.

heyashu.in · Ashutosh Anand Tiwari · 2026-08-24 · 6 pages · 11 min

LLM Embeddings Explained: How AI Understands Meaning & Context

Let's dig deep into LLM language, how AI understands similar words in different ways. Like I say, I ate apple at Apple Store, how does AI know the first apple is an eatable thing and the second Apple is the company name? So, it knows that. This is the main mystery we are going to crack.

Sometimes we only get confused, so how does a machine understand? How does NLP solve this issue? We already know about tokenization and token IDs and all, so let's go next.

Or whether that token makes sense or is just a random number. Like if **Dog** is represented with `8123`, does that make sense? **Grapes** are `8521`; are those two related? But actually, no. The numbers are near, so the answer is no, they are just dummy token ID numbers associated with those words/tokens.

So token ID doesn't contain meaning. Remember this, and no two IDs are related to each other. Then how?

Vectorization

It's the process of converting information into numerical vectors. It's like an array of numbers.

»Vectorization in large language models is **the process of turning words, sentences, or images into long lists of numbers so that a computer can understand their meanings and relationships.**«

So any information, text, document, words, images, anything can be converted into an array of numbers. But this is happening because computers can perform mathematical operations on top of it.

So suppose you need to rank fruits based on three properties: sweetness, size, and crunchiness. So for Apple, it will be `[.7, .4, .8]`, for Banana `[.8, .6, .2]`, and for Carrot `[.2, .5, .9]`. So this is just an example based on those three characteristics.

So, on certain dimensions, these numbers are converted into vectors. Like Apple here and other fruits are ranked on dimensions like sweetness, size, and crunchiness. So like this, other words can also have multiple dimensions, and vectorization happens based on that.

So this is done by embedding models, and the process is called vectorization. The embedding is a large array of numerical numbers. All companies have their own embedding models.

Modern embedding models use thousands of dimensions. Those dimensions are learned from the data, the vast data they got trained on. So Apple Store and apple fruits are identified easily.

So Apple with sweetness and crunchiness was talked about multiple times, and Apple with buying Mac devices and iPhone was discussed and represented multiple times, so dimensions are created based on all that data.

We will discuss more on it later about embeddings, but for now, we just discussed overall how it happens.

Embedding

An embedding in an LLM is a way to turn words, sentences, or pieces of text into a list of numbers that capture their actual meaning. So, embedding is a learned numerical representation of an item that captures useful relationships with other items.

So, for example:

King - `[.81, .32, .52]`

Queen - `[.79, .36, .48]`

Banana - `[.24, .91, .11]`

So, on this embedding, you can see King and Queen are relatable, but not with Banana. So, if we have thousands of dimensions, we can have relationships mathematically possible.

And this embedding happens based on the data we have and the relationships. Embeddings show relationships. So, associated words' embedding values will be together. Just remember it.

And this happens naturally based on data, based on some algorithms, and lots of GPUs are required to do this continuously on upcoming fed data. The model is not given these values by a human, that is clear now.

During training, the embedding values are learned. So, in the last word, it develops numerical relationships. Embedding is not the meaning of the words.

Embedding as coordinates

Let's try to visualize it in a 2D plot with only 2 dimensions, with this example.

These coordinates illustrate semantic clustering and geometric vector relationships in a 2D embedding space:

- **Fruits: Apple** (1, 2) and **Banana** (1.5, 2.3) cluster in the lower-left quadrant.
- **Programming Languages: JavaScript** (4, 8) and **Python** (4.5, 7.8) group near the upper-center.
- **Human Roles: Man** (7, 4) and **Woman** (6.5, 4.2) sit centrally on the right.
- **Royalty: King** (8, 7) and **Queen** (7.5, 7.2) sit in the upper-right.

Notes: More dimensions do not mean more intelligence. Higher dimensions can capture complex patterns, but they also require more storage and computation. And with each training, these numbers are adjusted.

Semantic similarity

Semantic similarity means measuring how close two pieces of text are in the same meaning. I mean, different kinds of statements but have the same meaning. Like Query A: How do I

center a div? Query 2: How can I align an HTML element in the middle of its parent? So, those both have similar meaning but different words.

In easy words, Semantic similarity in large language models is **a way for computers to check if two sentences mean the same thing, even if they use completely different words**.

But making this possible was not possible and was complex because of word combinations. But semantic similarities can be related if we use embeddings, and this is only possible because of embeddings. And note one thing: embedding of words is different from embedding of tokens, so we will discuss all of this later.

Embedding is the core thing for AI models. Learn from a research paper and learn about the embedding thing. It would be very fun to learn.

Embedding allows a system to compare broader meaning rather than only matching identical words.

COSine Similarities,

The process of achieving semantic similarity is called **cosine similarity**.

Cosine similarity **measures how close the meanings of two pieces of text are by looking at the angle between their arrow representations (vectors).**

Learn here: https://www.ibm.com/think/topics/cosine-similarity

If two vectors point in a similar direction, the represented concepts can be similar. If they point in unrelated directions, the concepts may be less similar.

Read the above-mentioned blog, you will learn more if you want to go deeper into the maths. I'm not going too deep into that, sorry ☹. I'm just thinking there is a factor that calculates vector direction, and based on that, semantic similarities get identified.

You can read here too: https://www.learndatasci.com/glossary/cosine-similarity/

Two Similarity Does Not Understand Truth

Two false statements can be semantically similar.

Two harmful instructions can be semantically similar.

Two texts can be close in topic but disagree completely.

»Embeddings capture relationships. They do not independently verify facts, intent, quality, or safety. Remember it.«

If you want to visualize the embedding, go to this website: https://projector.tensorflow.org/

Here I'm pasting two screenshots. See the relation in the embedding projector of **sun** and **JavaScript**, and its nearby embedded words.

Token Enmbedding

So never get confused: a token can also have an embedding, and a word can also have an embedding. So token ID has no meaning, as we know, but the token IDs can have embeddings of an array, and that embedding of that token

shows the relation between words and between tokens we can see.

So token IDs are meaningless, and embeddings are meaningful vectors.

So token embedding relates a meaningless token ID with a dense numerical vector. That vector becomes the model's initial representation of the token, because the representation may later change as the model processes the context.

»Token embedding **turns a piece of text into a list of numbers that a computer can use to understand meaning.**«

So a sentence like `I love JavaScript` is converted into tokens and token IDs. The token IDs are converted into embeddings like an array, and they relate to some other embeddings, and this forms a relationship.

But remember, it can change as the model processes the context. I mean, the way that token and word embeddings are used in a sentence with other words or tokens can change based on the context.

Positional embedding

So let's take an example where we say **Dog bites Man**, **Man bites Dog**. So the embedding for **bites** would also be the same for these two, but the context output will be different. How? And this all happens based on positional embeddings. So the order of the embedding, or you can say that the token or word position, also matters.

»Positional embedding is **a technique used by large language models (LLMs) to give words a sense of order and sequence in a sentence**.«

The model also needs information about the order or position before answering the query. Models therefore incorporate positional information using architecture-specific techniques.

The embedding tells the model which token is present. Positional information helps tell the model where that token appears, and the model can identify both the token and its order.

Token vs Text Embedding

Token embedding helps a language model to process a sequence internally. Text embeddings are often produced specifically for tasks such as search, retrieval, clustering, recommendation, classification, duplicate detection, etc. So when the task is about the order and all of the tokens, then token embedding matters, and when we search, the complete text embedding comes.

»The main difference is scope: **Token Embeddings turn individual pieces of words into numbers to help a model read text, while Text Embeddings (often called sentence or document embeddings) turn an entire sentence, paragraph, or document into a single mathematical summary to capture its overall meaning.**«

Think of it like looking at a book: a **Token Embedding** is a definition for a single syllable or word, while a **Text Embedding** is the summary printed on the back cover of the book.

And with the previous example, we understood that contextual and positional embeddings help words form relationships. A static embedding will not work overall.

Polysemy means **a single word has many different meanings depending on the sentence**.

How LLMs handle context

Now, from a long time, we are using word context, like order changes the context. With training, after context values can be changed, but let's understand how a model handles the context.

Java is a language.

Java is an island in Indonesia.

In a Transformer-based language model, the token first receives an initial token embedding, then the model processes it together with the surrounding tokens.

So suppose for our previous example, **Java** in both statements can have the same embedding `[.91] [.91]`. Then the model processes it together with the surrounding tokens. Through multiple layers, the representation becomes contextualized, and it all happens using Transformer architecture. We will discuss more about this later.

This initial word **Java** can become different in the second statement: **Java is an island in Indonesia.** The initial token may be the same, but when the model processes the complete sentence, its internal representation changes according to the context.

»Contextualization in a Large Language Model (LLM) means **giving the AI background information or clues so it understands what you mean, rather than just guessing based on a few isolated words**«

There is biasness also that can happen. Suppose the model was trained with one-sided data, so that can also happen because the data itself was biased.

Note: Modern LLMs use Hybrid Search, which includes keyword and embedding search for better efficiency.

These embeddings can be used in recommendation, clustering, multimodals classification duplicate detecttion and RAG also . We will remember these things and check them once we learn about them in more detail in the future.

Just keep in mind, an embedding is a general technique for representing information numerically.

Misconception

☒ An embedding is a dictionary of tokens

☒ An embedding is a numerical representation capturing learned relationships.

☒ Each dimension has one clear human meaning

☒ Meaning is distributed across many dimensions.

☒ Similar embeddings mean identical meaning

☒ They may indicate topic similarity, association, opposition, category membership, or shared context

☒ Embedding similarity proves a statement is true

☒ Similarity measures relatedness, not truth.

☒ One embedding model works equally well for every task

☒ Performance depends on: Language, Domain, Data type, Text length, Training objective, etc

☒ Visual clusters perfectly represent the original space

☒ Two-dimensional and three-dimensional projections lose information.

☒ A larger embedding dimension is always better

☒ Larger representations create trade-offs in storage, latency, cost, and quality

That's all for this episode. We will go more deep in the next one. Thanks, keep learning! ☒

Want me to write notes on a course?
Tell me the course, book, or research topic you want covered. If it helps learners, I'll consider adding structured digital notes for it in the garden.

Have a notes collection to feature here?
Do you have open notes you want featured in this digital garden? Let me know — we can collaborate and publish them for learners worldwide.

heyashu.in · Ashutosh Anand Tiwari · 2026-08-25 · 5 pages · 10 min

How LLMs Work: Neural Networks, Transformers & Attention Explained

So as we know, we are very smart, haha. Humans are smart, right? But how do we do things smartly? What do we use? Our mind, correct?

Correct. Now the question comes: machines are also becoming smart, isn't it? And so, it's all possible because of the computational mind of the machine or computer, and that is the **Neural Network**.

Now let's understand with an example: **The Pizza is __**. So the next words can be anything: **hot, cold, delicious, round, tasty**, etc. So which next word should come? That the neural network decides, and it's the responsibility of the neural network. The way our brain decides, in the same way the AI model's neural network decides the next word.

So when a sentence comes as input, that is broken into tokens, and those tokens are converted into embeddings. That array of embeddings is input into the neural network to get the next word.

As you can see in the above, all embeddings go as input into the neural network, and the next word is decided by the brain-like neural network. But how is the word decided? It's all the work of the neural network. Let's dig deep now into how words are decided.

So, the next generated word, including the sentence, is input into the neural network again as embeddings, so it guesses the next word. And it keeps going until it finishes. But when it ends, it all depends on the termination logic written in the LLM. A special end-of-sequence token can be used to stop generation; there can also be other stopping conditions. This is how, on top of the definition and explanation, the neural network is like a brain that decides the next word.

GPT in ChatGPT

The full form of GPT is **Generative Pre-trained Transformer**. What is this? So remember, it is always **Generative Pre-trained Transformer**.

What is Generative? It generates, it does not do retrieval. I mean, based on its knowledge, it generates. Ye ratta nahi marta, wo rattu tota nahi hai, ye sab samajh ke hume jawab deta hai.

Pre-trained means it is already trained, but what is this Transformer? And this is what is the real hero.

So, the Transformer is the main architecture that runs behind the neural network. Before it, different types of algorithms were being used, but after 2017, Google introduced the Transformer architecture through the research paper `Attention Is All You Need`, and that transformed everything.

»A Transformer in artificial intelligence is**a type of neural network design that reads entire sentences or blocks of data all at once instead of word by word**. It powers modern AI tools like ChatGPT, Claude, and Gemini.«

A Transformer is a type of architecture designed to process sequences of information using attention. It was introduced in 2017. Before Transformers, RNNs and LSTMs were being used.

The heart of the neural network is the **Transformer**, and the heart of the Transformer is **Attention**.

Attention Is All You Need PDF link:

By the end of all our learning, I hope we will be able to understand whatever is written in the Transformer architecture research paper, so fingers crossed. :)

Down, I have listed the author names of this paper. **Ashish Vaswani** https://www.linkedin.com/in/ashish-vaswani-99892181 avaswani@google.com

Noam Shazeer https://www.linkedin.com/in/noam-shazeer-73238914 noam@google.com

Niki Parmar https://www.linkedin.com/in/nikiparmar nikip@google.com

Jakob Uszkoreit https://www.linkedin.com/in/jakob-uszkoreit-6b586071 usz@google.com

Llion Jones https://www.linkedin.com/in/llion-jones-06752011 llion@google.com

Aidan N. Gomez https://www.linkedin.com/in/aidangomez aidan@cs.toronto.edu

Łukasz Kaiser https://www.linkedin.com/in/%C5%82ukasz-kaiser-6091391 lukaszkaiser@google.com

Illia Polosukhin https://www.linkedin.com/in/illia-polosukhin-77b6538 illia.polosukhin@gmail.com

Note: GPT is not related to LLM assistant ChatGPT. Basically, all models are GPT models (has a generative pre-trained transformer in it, OpenAI was the first movers to that named their assistant as ChatGPT, that's it.

Attention is all you need!

Let's do some attention to the name of the research paper. What is attention here? Why was this word used?

Attention means, from the given query, which words it needs to pay more attention to and how much?

So, in easy words, **Attention** in Artificial Intelligence (AI) is **a tool that helps computer models focus on the most important parts of data while ignoring the rest**.

For example: **The cat sat on the mat because it was tired.** We will discuss this next, but let's learn one more terminology: **Self-Attention**.

Self-attention is **a method that helps an AI understand the meaning of a word in a sentence by looking at all the other words around it at the same time**. It comes from the famous 2017 research paper titled **"Attention Is All You**

Need" by Google researchers. This paper introduced the **Transformer** model, which is the foundation of modern AI tools like ChatGPT.

So, in the above example, each token in the sentence looks at the other tokens in the same sentence to understand the relationship. Like the word `it` is more related to the `cat`, and this is called self-attention.

And what is magic in this? The magic is that before this Transformer in 2017, this revolution was not there, and relationships were missed with RNNs and other methods that were used in neural networks.

I hope I'm making you understand now!

In the above image, you can see the LLM's neural network structure. Before a token is embedded into the Transformer, all words are converted into tokens, and then those tokens are converted into embeddings. After token embedding, positional embedding happens, and token embedding + positional embedding get passed to the Transformer as input to predict the next word.

Token Embedding + Positional Embedding ⇒ goes to Transformer as input, and all mathematics happens. So, in the Transformer, the next thing that happens is **Layer Norm**, aka **Layer Normalization**.

Layer Normalization

Layer normalization (LayerNorm) is a technique used to keep the numbers flowing through a neural network at a more stable scale. So, in mathematical operations, numbers can become very large or very small, and calculations can become complex. Normalization happens to make it easier and more efficient, and this is what normalization is called **LayerNorm**.

Layer normalization is a tool in **Transformer AI** that **keeps numbers stable so the model can learn smoothly**.

What is Layer Normalization?

The Problem: As data moves through a Transformer, the numbers (called weights or activations) can become too big or too small.

The Fix: Layer normalization fixes this by adjusting the numbers inside each separate layer so they have a **mean (average) of 0** and a **variance (spread) of 1**.

The Goal: It stops the network from breaking or learning too slowly.

So, the question is: **When normalization happens, does the model lose the memory?** This is an open question. Go and do the research. [Answer you write here]

Next step is Attention

Attention

Before explaining this, I will tell you that this word is also known by multiple names, like **Multi-Head Attention, Masked Attention, Causal Self-Attention, and Attention**.

We learned about self-attention. What is that? Let's re-learn.

Self-attention is **a method that helps an AI understand the meaning of a word in a sentence by looking at all the other words around it at the same time**. It comes from the famous 2017 research paper titled **"Attention Is All You**

Need" by Google researchers. This paper introduced the **Transformer** model, which is the foundation of modern AI tools like ChatGPT.

If you see the above image, below the Layer Norm, **Multi-Head Attention** and **Causal Attention** are also written. So, let's discuss what that is.

Masked / Causal Selft Attention

So, AI checks the previous words. So, in easy words, I summarize:

Masked means hiding parts of the sentence.

Causal means causes come before effects (the past leads to the future).

Masked self-attention blocks the AI from seeing future words. The AI can only look at words that came *before* the current word.

A token can look at itself and the tokens before it, but never the future tokens. **Causal** is important because information is only allowed to flow in one direction: **past to present**.

Multi-head

Lots of parallel processes are running to do the self-attention block or part, and this is called **Multi-Head Attention**.

Multi-head attention is a core part of modern **AI models** like ChatGPT that **lets the computer look at many different things in a sentence at the same time**.

Multi-head causal self-attention allows every token to gather information from previous tokens using multiple attention heads, while preventing it from looking into the future.

Residual /Skip connection

Instead of completely replacing the old information with the output of a layer, we add the original input back to the layer's output. See the above image, and the `+` icon and arrow come and attach from the top.

A **residual connection** (also called a skip connection) is a direct shortcut in an AI model that lets input data bypass a processing layer and add itself to the layer's output.

So, for example, `x = [1, 2, 3]` was there, and after one layer of attention, it becomes `Attention(x) = [.2, -.5, .7]`. Now, instead of passing this forward, we add the original input:

```
x + Attention(x)
```

giving the result as `[1.2, 1.5, 3.7]`.

You can think of each layer as adding a useful update rather than rebuilding the entire representation from scratch.

A Transformer layer does not throw away what it already knows. It learns an update and adds that update on top of the existing representation.

And then **Layer Norm** happens, and then **FFN (Feed-Forward Network)** happens.

Feed Forward Network (FFN)

As we know, during self-attention, tokens interact with each other, right? Like if I'm sitting on the bank of a river, **bank** finds its meaning after looking at the other words. But during FFN, each token is processed independently.

Attention lets tokens communicate, FFN lets each token think/process what it learned.

A Feed-Forward Network (FFN) is **a key building block inside a Transformer AI model that processes information and helps the AI think more deeply about the meaning of words after the attention step**.

See the above image. The hidden layer is the FFN that tweaks the output embedding after an attention phase.

I don't know much; I'm learning and writing these lines. Maybe in the future, I will learn more and rewrite or change it. So, dear reader, it's your responsibility. Please be curious and edit or update the notes if required, and keep learning.

Okay, so after FFN, **normalization** happens, and then the final steps, **Linear and Softmax**, take place. Let's learn that.

Linear and Softmax

So what the words are likely to be as the result and prediction, those words with their embedding values get a percentage kind of probability. So suppose the new words **cat, dog, car, and pizza** are finalized, so these will get a percentage of probability. The highest one gets the entry as the result, the predicted word.

»A **linear layer** in a Transformer AI acts like a math machine that changes the size and shape of information, while a **SoftMax layer** acts like a referee that turns raw numbers into percentages that add up to 100%.«

As a user of AI, I know this becomes complicated for you, but if you love maths and research and all, you can go and research and learn how the formulas are used to make this possible.

Now it's time to learn and visualize these mathematics. Let's visit this website: https://bbycroft.net/llm

If you open the website, you will see the same image, and you can use it to visualize each step we discussed till now. I know it's tough, and it was tough for me too. Still, I'm not able to understand so many things, so we need to slowly adapt to these things and all. Read this also : https://github.com/karpathy/nanoGPT

That's all for today. Next, we will try to understand how models get trained. Happy learning! Bye-bye! ⊠

Want me to write notes on a course?

Tell me the course, book, or research topic you want covered. If it helps learners, I'll consider adding structured digital notes for it in the garden.

Have a notes collection to feature here?

Do you have open notes you want featured in this digital garden? Let me know — we can collaborate and publish them for learners worldwide.